Azure Arc Onboarding at Enterprise Scale: A Complete Deployment Guide

An expert deployment guide for connecting Windows Server and Linux machines to Azure Arc at scale: landing zone and RBAC, service principals versus managed identities, proxy, Private Link and Arc gateway, the Connected Machine agent and its local security controls, automated onboarding with PowerShell, Group Policy, Configuration Manager and Ansible, Azure Policy governance, Azure Monitor Agent, Defender for Cloud, troubleshooting, verification and offboarding.

11 phases | 41 steps | Tick each item when its "done when" criterion is true

Phase 1 Plan the rollout: scope, inventory, pilot and cost

about 3–5 days + 30-day pilot

- [ ] **1.1 Draw the support boundary before you inventory**
  Every candidate machine is listed with OS, version, architecture, hosting platform and owner, and is checked against the Connected Machine agent support table, including the end-of-new-agent date for its OS
- [ ] **1.2 Run a pilot with written success criteria**
  A pilot plan exists with objectives, scope (machine count and types), success metrics, a rollback procedure and an escalation path, and runs for at least 30 days
- [ ] **1.3 Separate free control-plane features from paid services**
  A cost model lists which paid services (Defender for Servers, Azure Monitor ingestion, Update Manager, machine configuration, ESU) will be enabled per machine group, and which machines qualify for exemptions
- [ ] **1.4 Check the Microsoft Entra directory quota for large estates**
  The tenant's directory object usage and limit are known, and there is headroom for one new object per Arc server plus growth

Phase 2 Landing zone: subscriptions, resource groups, tags and resource providers

about 2–3 days

- [ ] **2.1 Decide where Arc resources live**
  A written rule maps each machine group (site, environment, owner) to a management group, subscription, resource group and Azure region
- [ ] **2.2 Define the tag taxonomy before the first production wave**
  Mandatory tags (for example environment, site, owner, criticality, patch ring) are documented with allowed values, and the onboarding script sets them at connect time
- [ ] **2.3 Register the resource providers in every target subscription**
  `Microsoft.HybridCompute`, `Microsoft.GuestConfiguration` and `Microsoft.HybridConnectivity` are registered everywhere Arc servers will land, plus `Microsoft.Compute` (and `Microsoft.AzureArcData` if SQL Server will be Arc-enabled)

Phase 3 RBAC: least privilege for onboarding, operations and extensions

about 1–2 days

- [ ] **3.1 Use the two Arc built-in roles for what they were designed for**
  Azure Connected Machine Onboarding is held only by onboarding identities; Azure Connected Machine Resource Administrator is held by the Arc operations group, through PIM, at resource group scope
- [ ] **3.2 Treat extension rights as server admin rights**
  Every principal with Contributor, Owner or Resource Administrator on Arc server scopes is reviewed as an indirect local administrator of those servers, and Tier 0 servers sit in their own scope
- [ ] **3.3 Map the hybrid identity model for server access**
  The access model states which access stays with Active Directory (logon, local admin via LAPS) and which moves to Azure RBAC (management, extensions, policy), and who holds each

Phase 4 Service principals and managed identities

about 1 day

- [ ] **4.1 Create a dedicated onboarding service principal per landing zone**
  One service principal per target scope (subscription or resource group) exists with only Azure Connected Machine Onboarding, its secret expiry is recorded, and the secret is stored in a vault
- [ ] **4.2 Prefer managed identity for the automation host where possible**
  Automation that runs on an Azure VM or an Arc-enabled server (for example an Ansible control node) authenticates with its managed identity, which holds the Onboarding role, instead of a stored secret

- [ ] **4.3 Understand the per-server managed identity Arc creates**
  Operations know that each server has a system-assigned identity with a 90-day certificate, where it is stored, who can request tokens, and what happens after a long disconnection

Phase 5 Proxy and network requirements: direct, proxy, Private Link or Arc gateway about 3–10 days (mostly firewall change control)

- [ ] **5.1 Choose a connectivity pattern per network zone**
  Each zone (DMZ, data centre, branch, other cloud) has one pattern — direct, enterprise proxy, Private Link scope or Arc gateway — with the reasoning documented
- [ ] **5.2 Hand the firewall team the official endpoints and service tags**
  Firewall and proxy rules allow the documented URLs or service tags for the clouds and features in use, including regional Entra endpoints, and nobody filters on single resolved IPs
- [ ] **5.3 Configure the agent's proxy and bypass rules**
  Proxy settings are set through `azcmagent config` (not system-wide environment variables), bypass values match the network pattern, and `azcmagent show` reports the effective configuration
- [ ] **5.4 Build the Private Link scope and DNS correctly (if used)**
  A Private Link scope with a `hybridcompute` private endpoint exists, `privatelink.his.arc.azure.com` and `privatelink.guestconfiguration.azure.com` resolve to private IPs from the servers, and public network access is disabled once all servers are associated
- [ ] **5.5 Enforce TLS 1.2+ and the agent's cipher suites**
  Every server negotiates TLS 1.2 or 1.3 with the endpoints, and Windows servers on agent 1.56+ have at least one of the required cipher suites enabled

Phase 6 The Azure Connected Machine agent: components, local controls and lifecycle about 1–2 days

- [ ] **6.1 Know the components, services and logs**
  Runbooks list the agent services, folders and log files for Windows and Linux, and temporary antivirus exclusions for installation are agreed with security
- [ ] **6.2 Lock down sensitive servers locally: allowlists, monitor mode, no remote scripts**
  Tier 0 and regulated servers have an extension allowlist or monitor mode set locally, Custom Script and Run Command are blocked where not needed, and the configuration is recorded per server class
- [ ] **6.3 Make golden images and clones safe**
  Images may contain the installed agent but never a connected one; the build pipeline connects each machine after cloning, and a runbook exists for 429 errors from cloned identities
- [ ] **6.4 Keep the agent inside its support window**
  Automatic agent upgrade (or Microsoft Update, WSUS or the Linux repositories) keeps every agent within the last year of releases, and an Azure Advisor alert flags outdated agents

Phase 7 Onboard Windows and Linux servers: choose the method per estate about 1–4 weeks in waves

- [ ] **7.1 Pick the onboarding method per machine group**
  A matrix maps each machine group to a supported method (script, PowerShell, Group Policy, Configuration Manager, Ansible, VMware vSphere, AWS connector or Arc Setup) and names the identity it uses
- [ ] **7.2 Onboard Windows servers with Group Policy or Configuration Manager**
  The GPO or task sequence targets only the intended OU or collection, the agent MSI comes from an internal share or the Download Center, and the GPO is disabled after the wave is confirmed
- [ ] **7.3 Onboard Linux servers with Ansible or the service principal script**
  Linux waves run from a playbook in source control with a managed identity or a vaulted service principal, the Microsoft package repository is reachable, and reruns are idempotent
- [ ] **7.4 Size and sequence the production waves**
  Waves are defined (for example 10 → 50 → 200 → the rest) with entry criteria (network test passed, owners informed) and exit criteria (all Connected, tags correct, no failed extensions)

Phase 8 Automated onboarding with PowerShell about 2–4 days to build, minutes per wave

- [ ] **8.1 Use Connect-AzConnectedMachine for interactive and remoting waves**
  Operators can onboard a list of servers through PowerShell remoting with `Connect-AzConnectedMachine`, signed in with their own (PIM-activated) identity, with the output captured per machine

- [ ] **8.2 Wrap the generated script: pre-checks, secret at run time, tags as parameters**
  The wrapper verifies OS, architecture and admin rights, runs `azcmagent check`, retrieves the onboarding secret at run time, passes tags and network settings as parameters, and exits non-zero with the AZCM code on failure
- [ ] **8.3 Make reruns idempotent and safe**
  Running the pipeline twice against the same machine does not create duplicates, does not reconnect healthy machines, and repairs only documented failure states
- [ ] **8.4 Deploy extensions and settings as code after onboarding**
  Post-onboarding configuration (extensions, Private Link association, automatic upgrade) is applied by policy or by Bicep, CLI or PowerShell from source control, not by hand in the portal

Phase 9 Govern with Azure Policy about 2–3 days, then continuous

- [ ] **9.1 Assign the platform policies for connectivity, agent lifecycle and tags**
  Policies for Private Link, automatic agent upgrade and mandatory tags are assigned at the right management group with remediation, and compliance is reviewed weekly
- [ ] **9.2 Use machine configuration for in-guest baselines**
  In-guest audit policies (security baseline, local Administrators membership, TLS settings) are assigned to Arc servers where guest configuration is enabled, and their cost is accounted for
- [ ] **9.3 Hand patching governance to the Update Manager framework**
  Periodic assessment and scheduled patching policies for Arc servers are assigned as described in the Update Manager guide, using the patch ring tag set at onboarding

Phase 10 Azure Monitor and Defender for Cloud about 2–4 days

- [ ] **10.1 Deploy Azure Monitor Agent and data collection rules through policy**
  Policies install AMA on Windows and Linux Arc servers and associate them with the intended data collection rules; the Heartbeat table shows every connected server with its Arc resource ID
- [ ] **10.2 Alert on disconnected servers and outdated agents**
  A Resource Health alert fires when an Arc server goes from Available to Unavailable, and an Advisor alert fires for outdated agents; both route to the server owner
- [ ] **10.3 Protect Arc servers with Defender for Cloud**
  Defender for Servers is enabled at the intended plan for the Arc server subscriptions (or by tag at resource level), Defender for Endpoint integration is on, and Arc servers appear in the Defender for Cloud inventory

Phase 11 Troubleshooting, verification and offboarding about Ongoing

- [ ] **11.1 Verify every wave from the server and from Azure**
  For each wave: `azcmagent show` reports Connected on every machine, Resource Graph shows the same count with correct tags and region, and no extension is in a failed state
- [ ] **11.2 Triage onboarding failures by AZCM code**
  First-line runbooks map the common AZCM codes to a cause and a fix, and verbose logs are collected for everything else
- [ ] **11.3 Troubleshoot extensions and disconnected servers**
  Failed extensions are diagnosed from the extension manager logs and the Activity Log; disconnected and expired servers have a documented recovery path
- [ ] **11.4 Offboard, rename and move servers cleanly**
  Decommissioning, renaming or moving a server follows one runbook: remove extensions, disconnect, then uninstall (or reconnect), and the resource and its identity are gone from Azure
- [ ] **11.5 Next step: build the patch framework on top of Arc**
  The onboarded estate has the patch ring tags, `Microsoft.Compute` registration and update-source connectivity that the Update Manager framework needs, and its planning phase has an owner

Owner	Change / ticket	Date completed

Original checklist by Tenant Recipes, an independent project not affiliated with Microsoft or any vendor. Every step, its actions, pitfalls and the official sources it was checked against are on https://tenantrecipes.com/guides/azure-arc-onboarding-enterprise-scale.html. Time estimates are our own planning figures. Confirm against the linked sources and test in a pilot before production.